

method ❹, we fill the background, draw the message field, and draw the buttons. This example scene does very little, so we don't need to write our own `enter()`, `update()`, and `leave()` methods. Calls to these methods will be handled by methods of the same names in the `Scene` base class, and those methods don't do anything—they simply execute a `pass` statement.

The two other scene files are *SceneB.py* and *SceneC.py*. The only differences are the titles shown, the buttons drawn, and the effects of clicking the buttons to transfer to the appropriate new scene.

Rock, Paper, Scissors Using Scenes

Let's build an alternate implementation of the Rock, Paper, Scissors game using the scene manager. To the user, the game will work exactly the same way as the earlier state machine version. We'll build a Splash scene, a Play scene, and a Results scene.

All of the source code is available, so I won't go through every Python file. The Splash scene is just a background picture with a Start button.

When the user presses the Start button, the code executes

`goToScene(SCENE_PLAY)` to transfer to the Play scene. In the Play scene, the user is presented with a set of images (rock, paper, and scissors) and asked to choose one. Clicking an image transfers control to the Results scene. [Listing 15-4](#) contains the code of the Play scene.

File: RockPaperScissorsWithScenes/ScenePlay.py

```
# The Play scene
# The player chooses among rock, paper, or scissors

import pygwidgets
import pyghelpers
import pygame
from Constants import *
import random

class ScenePlay(pyghelpers.Scene):
    def __init__(self, window):
```